

BEST ERP OF 2027

A Realistic, Phased Master Plan for Building an S/4HANA-Class ERP

Prepared for Lokman | July 31, 2026

Reality check before you read further:

SAP S/4HANA represents roughly 50 years and tens of thousands of engineer-years of cumulative work. No team builds an equivalent from zero in a year. This plan is written to actually ship something real: a phased roadmap that gets a working, production-grade ERP into users' hands fast (Phase 1), then grows it into the HTAP / Kafka / AI-copilot architecture you specified (Phases 2–3) as load and funding justify it. Every technology choice below resolves the “or” options in your spec with a concrete recommendation and the reasoning behind it.

1. Executive Summary

Your specification is architecturally sound — HTAP database, event-driven microservices, Kafka-triggered MRP, Kubernetes autoscaling, and an AI copilot are all legitimate parts of a modern ERP. The risk isn't the target architecture; it's sequencing. Building the full stack (distributed HTAP database, Kafka, microservices, K8s autoscaling, five core modules, and an LLM copilot) simultaneously before a single user has logged in is how ERP rebuilds stall for years and burn budgets with nothing shipped.

This plan restructures the work into three phases so that Phase 1 alone is a usable, sellable product, and each later phase is justified by real, measured load rather than anticipated load:

- **Phase 1 — Foundation & MVP (Months 1–8):** modular monolith, tuned PostgreSQL, core Security/FICO/MM modules, single Kubernetes cluster. Goal: a real ERP running for a pilot customer or internal pilot group.
- **Phase 2 — Core Platform Expansion (Months 9–18):** add SD and PP, introduce Kafka and the Saga pattern, extract the highest-load services out of the monolith, add basic MRP.
- **Phase 3 — Enterprise Scale & Intelligence (Months 19–30+):** migrate hot-path data to an HTAP store, stand up Live MRP on Kafka streams, add the AI copilot, and prove 1000+ concurrent users under real load testing.

The remainder of this document details the architecture decisions, module sequencing, team plan, budget ranges, and risk register for each phase.

2. Phased Roadmap Overview

Phase	Timeline	Primary Goal	Team Size	Exit Criteria
Phase 1: Foundation	Months 1–8	Working MVP: Security, FICO core, MM basics	8–12	Pilot customer live; GL reconcile <300ms p95 API latency at 50 concurrent users
Phase 2: Core Platform	Months 9–18	Add SD + PP, event streaming, Saga pattern	16–22	300+ concurrent users sustained; Kafka-driven MRP recalculation <5s; zero unreconciled Sagas in staging soak test
Phase 3: Enterprise Scale	Months 19–30+	HTAP migration, Live MRP, AI copilot, 1000+ users	28–38	1000+ concurrent users at <500ms p95 under load test; autoscaling verified in chaos test; copilot answers 90%+ of scripted queries correctly

3. Phase 1 — Foundation & MVP (Months 1–8)

3.1 Database Layer

Recommendation: PostgreSQL 16+ (managed or self-hosted with Patroni for HA), not TiDB or SingleStore yet.

- PostgreSQL's MVCC already delivers non-blocking reads/writes for the concurrency levels Phase 1 needs (dozens to low hundreds of users). TiDB/SingleStore add distributed-systems operational overhead (multi-node Raft, PD placement, rebalancing) that isn't justified until you're past ~500 concurrent users or need cross-region writes.
- PgBouncer in transaction-pooling mode in front of Postgres from day one — this is cheap insurance against connection exhaustion and should never be skipped, even at low scale.

- Redis for session storage, active product/customer catalogs, and rate limiting. Use Redis Cluster mode only when a single node's memory or throughput becomes the bottleneck.
- Every write path uses real BEGIN/COMMIT/ROLLBACK transactions. No distributed Saga is needed yet because Phase 1 is a modular monolith with one database — Sagas solve a problem (distributed transactions across services) that doesn't exist until Phase 2's service extraction.

3.2 Backend

Recommendation: Go, structured as a modular monolith (not microservices) for Phase 1.

- Go over Java/Spring Boot: lower memory footprint per instance, faster cold starts (matters once you're autoscaling in Phase 3), and a concurrency model (goroutines/channels) that maps naturally onto the Kafka consumer groups you'll add in Phase 2.
- “Modular monolith” means: one deployable binary, but internally organized into strict packages per domain (security, fico, mm, sd, pp) with enforced internal API boundaries (no direct cross-package DB access). This gives you microservices-style separation of concerns without the operational tax of running 10 separate services on day one.
- This also directly enables the Phase 2 “strangler fig” migration: because module boundaries are already clean, you extract MM or FICO into its own service later by moving a package, not by rewriting it.

3.3 Frontend

Recommendation: React with a component library seeded from day one, even as a single frontend app (micro-frontends come in Phase 2–3 once multiple teams need to ship independently).

- Build the design system as its own versioned package (e.g. an internal npm package) from the very first screen. Retrofitting a shared component library after 5 modules have shipped their own buttons and tables is a multi-month tax — avoid it by starting centralized.
- Defer the AI copilot's execution capability (“run this query for me”) to Phase 3; ship a simple contextual help/search assistant in Phase 1 if you want an early AI presence, since giving an LLM write/query access before RBAC and audit logging are hardened is a real security risk.

3.4 Module Sequence (unchanged from your spec — this ordering is correct)

- Security Core: authentication, RBAC, and audit logging (who changed what, when, previous value) built first and enforced by middleware on every subsequent module — retrofitting audit logging later means re-touching every table.
- FICO core: General Ledger, Accounts Payable/Receivable, Asset Accounting. Every module built after this posts to the GL through one internal “posting service” interface — this is the single most important architectural rule in the whole system, because it's what makes MM/SD/PP transactions financially consistent.
- MM basics: inventory tracking, purchase orders, goods receipt/issue — enough to support a real pilot customer's day-to-day operations.

3.5 Infrastructure

- Docker Compose for local dev; a single managed Kubernetes cluster (EKS/GKE) for staging + production from month 1, but with 1–2 node pools and no custom autoscaling logic yet — just standard Horizontal Pod Autoscaler on CPU/memory.
- NGINX ingress controller as load balancer; defer the AWS ALB / multi-region discussion to Phase 3.

4. Phase 2 — Core Platform Expansion (Months 9–18)

Phase 2 begins only after Phase 1's pilot has run long enough to produce real usage data — that data, not guesswork, should decide which modules get extracted into services first.

- **Add Sales & Distribution (SD):** customer orders, pricing, shipping, billing — wired into the same GL posting interface established in Phase 1.
- **Add Production Planning (PP):** work centers, Bill of Materials, Routings, and a first-cut MRP batch job (nightly/hourly recalculation, not yet real-time).
- **Introduce Apache Kafka:** start with 2–3 topics (inventory-changes, order-events, mrp-triggers) rather than event-sourcing everything at once. Kafka's payoff in Phase 2 is decoupling MM/SD/PP from each other, not yet real-time MRP — that's Phase 3.
- **Extract 1–2 services from the monolith:** using the module boundaries built in Phase 1, pull out the highest-load module (commonly Inventory/MM or Pricing) into its own service. This is where the Saga pattern actually becomes necessary: any transaction that now spans the monolith and an extracted service needs compensating transactions, since a single DB transaction can no longer cover it.
- **Kubernetes:** introduce KEDA or custom HPA metrics (Kafka consumer lag, queue depth) instead of just CPU, since event-driven services scale on different signals than the request/response monolith.

Exit criteria for Phase 2: SD and PP live with a pilot customer doing real production scheduling; Kafka-based MRP recalculates within seconds of a factory-floor consumption event (not yet real-time, but no longer nightly); a Saga soak test in staging shows zero orphaned/unreconciled distributed transactions after 72 hours of synthetic load.

5. Phase 3 — Enterprise Scale & Intelligence (Months 19–30+)

This is where the full ambition of your original spec is realized — but now backed by 18 months of real production data about actual query patterns, actual concurrency, and actual bottlenecks, instead of assumptions.

- **HTAP migration:** migrate the hottest analytical + transactional tables (typically inventory, GL line items, order lines) to TiDB or SingleStore. Recommendation: TiDB if you want MySQL-wire-protocol compatibility and open-source flexibility; SingleStore if you want the fastest turnkey HTAP with strong vendor support and are comfortable with a commercial license. Do not migrate the entire database — keep low-volume reference/config tables in Postgres to avoid unnecessary operational surface area.
- **Live MRP:** rebuild the MRP engine as a Kafka Streams (or Flink) application that consumes material-consumption events and recalculates affected demand plans in milliseconds, replacing the Phase 2 batch/near-real-time version.
- **AI Copilot:** integrate an LLM (via API) with tool-calling access scoped by the same RBAC engine built in Phase 1, so the copilot can only query/act on data the logged-in user is already permitted to see. Start with read-only natural-language query (“show me the delay impact of this PO”) before granting it any write/execute capability, and log every copilot action through the same audit pipeline as human actions.
- **Kubernetes at scale:** multi-node-pool cluster with KEDA-driven autoscaling tied to Kafka lag and DB connection pool saturation; scheduled pre-scaling for known peak windows (month-end close, quarter-end).
- **Load balancing:** AWS ALB (or NGINX Ingress with multiple replicas behind a cloud LB) distributing 1000+ concurrent users across node pools, validated with actual load-testing tools (k6, Locust, or Gatling) before declaring it production-ready — not just architected on paper.

Exit criteria for Phase 3: a load test sustains 1000+ concurrent power users at under 500ms p95 API latency; a chaos test (killing pods/nodes mid-transaction) shows correct Saga rollback/compensation with no data corruption; the AI copilot correctly answers a scripted set of realistic operational questions with >90% accuracy under human review.

6. Technology Decision Matrix

Your spec left several choices open (“X or Y”). Here is a concrete recommendation for each, with the deciding factor.

Decision	Options Given	Recommendation	Deciding Factor
Database	TiDB / SingleStore / tuned Postgres	Postgres (P1) → TiDB or SingleStore (P3)	Don't pay distributed-systems tax before load requires it
Backend language	Go / Java (Spring Boot)	Go	Lower footprint, faster autoscale cold-start, concurrency model fits Kafka
Architecture style	Microservices / modular monolith	Modular monolith → strangler-fig to microservices	Ship faster in P1; extract services only where load data justifies it
Frontend framework	React / Vue.js	React	Larger enterprise component ecosystem (incl. shadcn/ui-style libraries) for a Fiori-like design system
Load balancer	NGINX / AWS ALB	NGINX Ingress (P1–2) → AWS ALB or both (P3)	NGINX is cloud-agnostic and sufficient until multi-region is needed

7. Team Composition & Hiring Plan

Role	Phase 1	Phase 2	Phase 3
Backend engineers (Go)	4	7	10
Frontend engineers (React)	2	3	5
Database / platform engineer	1	2	3
DevOps / SRE (K8s, Kafka)	1	2	4
QA / test automation	1	2	3
Product / functional (ERP domain expert)	1	2	3
ML / AI engineer (copilot)	0	1	2
Engineering manager / architect	1	1	2
Total	11	20	32

Note: an ERP domain expert (someone who has implemented SAP, Oracle NetSuite, or Dynamics before) is not optional. Financial posting rules, tax logic, and inventory valuation methods have decades of accumulated edge cases; underestimating this is the single most common reason in-house ERP builds fail functionally even when the engineering is solid.

8. Budget Estimate (Rough Ranges, USD)

These are order-of-magnitude planning ranges based on the team sizes above at typical fully-loaded engineering costs; actual figures depend heavily on region and whether roles are contracted or hired full-time.

Phase	Duration	Team Cost (fully loaded)	Infra / Tooling / Licensing	Phase Total (approx.)
Phase 1	8 months	\$1.3M – \$1.9M	\$40K – \$80K	\$1.35M – \$2.0M
Phase 2	10 months	\$2.8M – \$3.8M	\$100K – \$200K (Kafka, K8s scale)	\$2.9M – \$4.0M
Phase 3	12+ months	\$5.0M – \$6.5M	\$300K – \$600K (HTAP licensing, LLM API, load-test infra)	\$5.3M – \$7.1M
Total (30 months)	—	—	—	\$9.5M – \$13M

9. Risk Register

Risk	Likelihood	Impact	Mitigation
Scope creep: trying to build all 3 phases at once	High	Critical — most common cause of ERP rebuild failure	Enforce phase exit criteria (Section 2) as hard gates before starting the next phase
Underestimating FICO/tax/compliance complexity	High	High — wrong numbers erode customer trust immediately	Hire a domain expert in month 1, not after problems surface
Premature migration to distributed HTAP database	Medium	High — operational overhead without proven need	Migrate only when Phase 1/2 load testing shows Postgres is the bottleneck
AI copilot given write access before RBAC/audit hardened	Medium	Critical — security and compliance exposure	Read-only copilot until Phase 3; all copilot actions routed through existing audit pipeline
Kafka introduced before there's a real event volume to justify it	Medium	Medium — added complexity, slower delivery	Defer Kafka to Phase 2; start with 2–3 topics, not full event-sourcing
Talent gap: too few engineers with Go + Kafka + K8s experience	Medium	Medium	Budget for training/ramp time in Phase 2 hiring plan; consider Java/Spring Boot fallback if Go hiring stalls

10. Success Metrics by Phase

- Phase 1: pilot customer processes real GL/MM transactions daily for 4+ consecutive weeks with zero unreconciled ledger entries; p95 API latency under 300ms at 50 concurrent users.

- Phase 2: MRP recalculation latency drops from nightly batch to under 5 seconds after a factory-floor event; zero orphaned Saga transactions across a 72-hour soak test; SD order-to-cash cycle fully traceable back to GL.
- Phase 3: sustained 1000+ concurrent users at under 500ms p95 latency in load testing; automatic scale-up/down verified during simulated month-end close; AI copilot answers a 50-question operational test set with >90% accuracy, verified by a human reviewer.

11. Immediate Next Steps (First 90 Days)

- Days 1–30: hire the domain expert and lead architect; stand up the Postgres + PgBouncer + Redis + K8s baseline; finalize the internal module-boundary contracts (Security, FICO, MM) before writing feature code.
- Days 31–60: build Security Core (auth, RBAC, audit logging) end-to-end and demo it — every later module depends on this being solid.
- Days 61–90: build the GL posting interface and a minimal AP/AR flow; get one real transaction (e.g. a test purchase order) flowing from MM through to a GL entry, proving the “everything maps back to the GL” rule works before any other module is built on top of it.